Table of Contents

Introduction **2**

Logging **2**

Install Serilog *2*

Start Logging *4*

Filtering Logging Messages *7*

Error Handling **9**

Error Handling - IItemRepository *9*

Error Handling - ItemRepository *10*

Error Handling - ItemController *13*

Demo: Error handling and logging

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT and Gemini.

Introduction

In this demo, we'll delve into the implementation of error handling and logging. Effective logging and error handling are critical for maintaining and troubleshooting web applications.

- We start with installing Serilog. Serilog, a powerful third-party logging library, offers structured logging, allowing for detailed and organised log data.
- We'll demonstrate how to use and configure logger, set up logging, filter log messages.
- We will handle errors gracefully across different components of our application.

Logging

For logging, there are two popular approaches: the built-in Microsoft Extensions Logging (MEL) and the third-party library Serilog. Serilog is the reference example of a Structured Logging system, which automatically handles the contents of a log and puts them into a structured, easily queryable format. We use Serilog as it offers more powerful routing, formatting, and destination possibilities.
Read more:
https://onloupe.com/blog/serilog-vs-mel/
https://www.sumologic.com/glossary/structured-logging/

Install Serilog

Open the terminal and run:

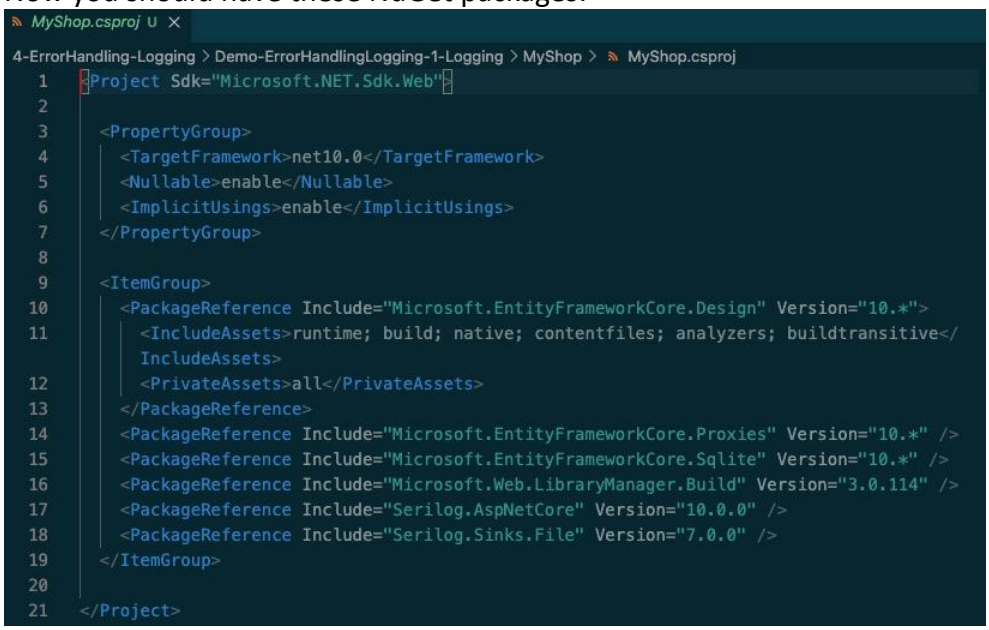
```
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.File
```

Serilog.AspNetCore
For modern .NET web applications and APIs, this is the primary package you should use. It integrates Serilog into the built-in .NET logging framework so you can continue using standard ILogger interfaces. It replaces older, lower-level packages by automatically including essential features:

- Built-in console logging (Serilog.Sinks.Console is included automatically).
- Integration with WebApplicationBuilder.
- HTTP request logging, which replaces verbose default ASP.NET logs with clean, single-line structured logs.

Serilog.Sinks.File This package provides a sink for logging to a text file. A "sink" is a destination where log events are sent. It allows you to persist log events to a file on disk, which is essential for production environments where you need to retain data for historical analysis, debugging, or auditing. Configuration options allow you to specify the file path, size limits, and rolling policies (e.g., creating a new log file daily).

Now you should have these NuGet packages:

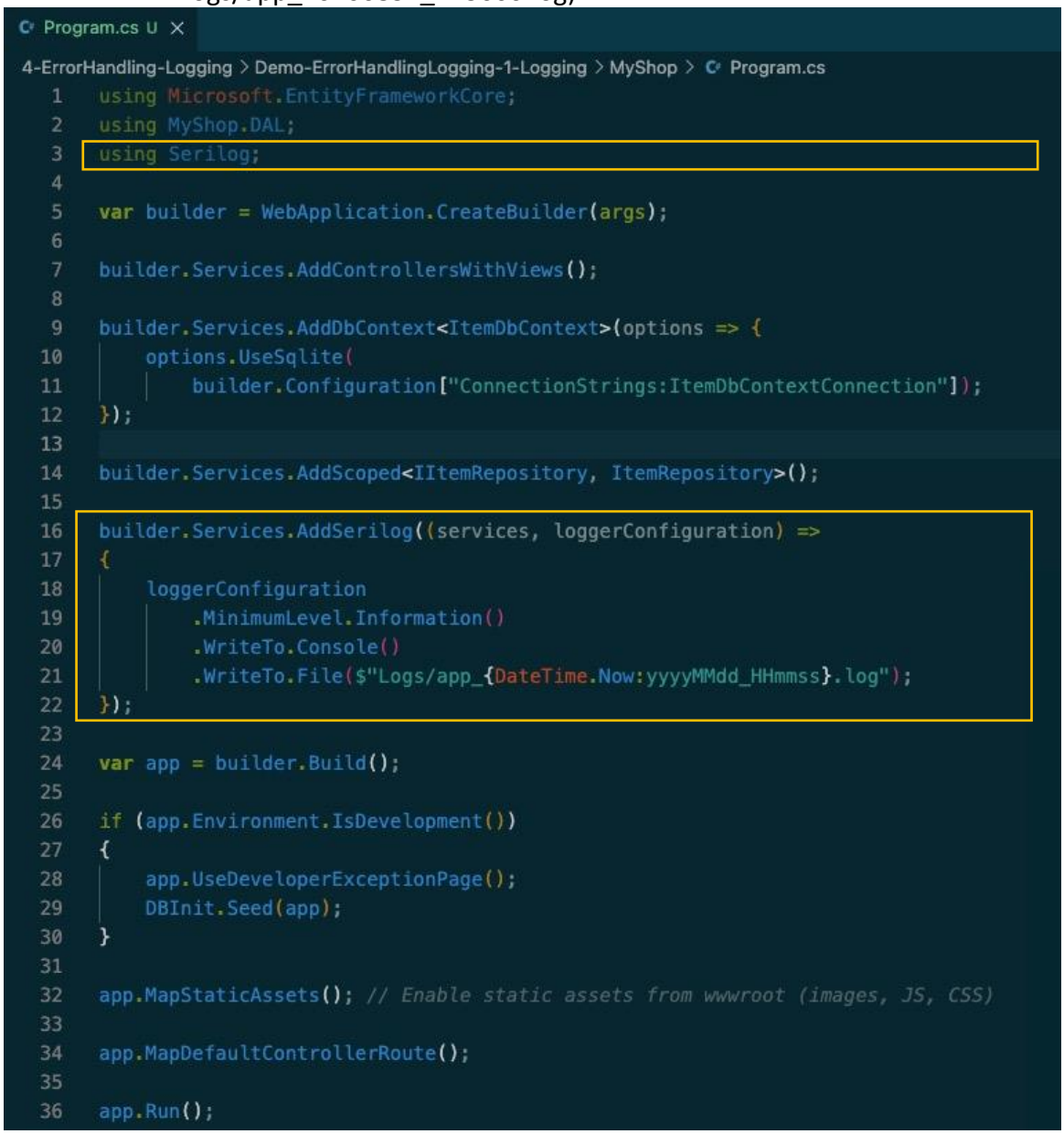


Start Logging

Configure Logging in Program.cs

Open Program.cs and add the Serilog configuration:

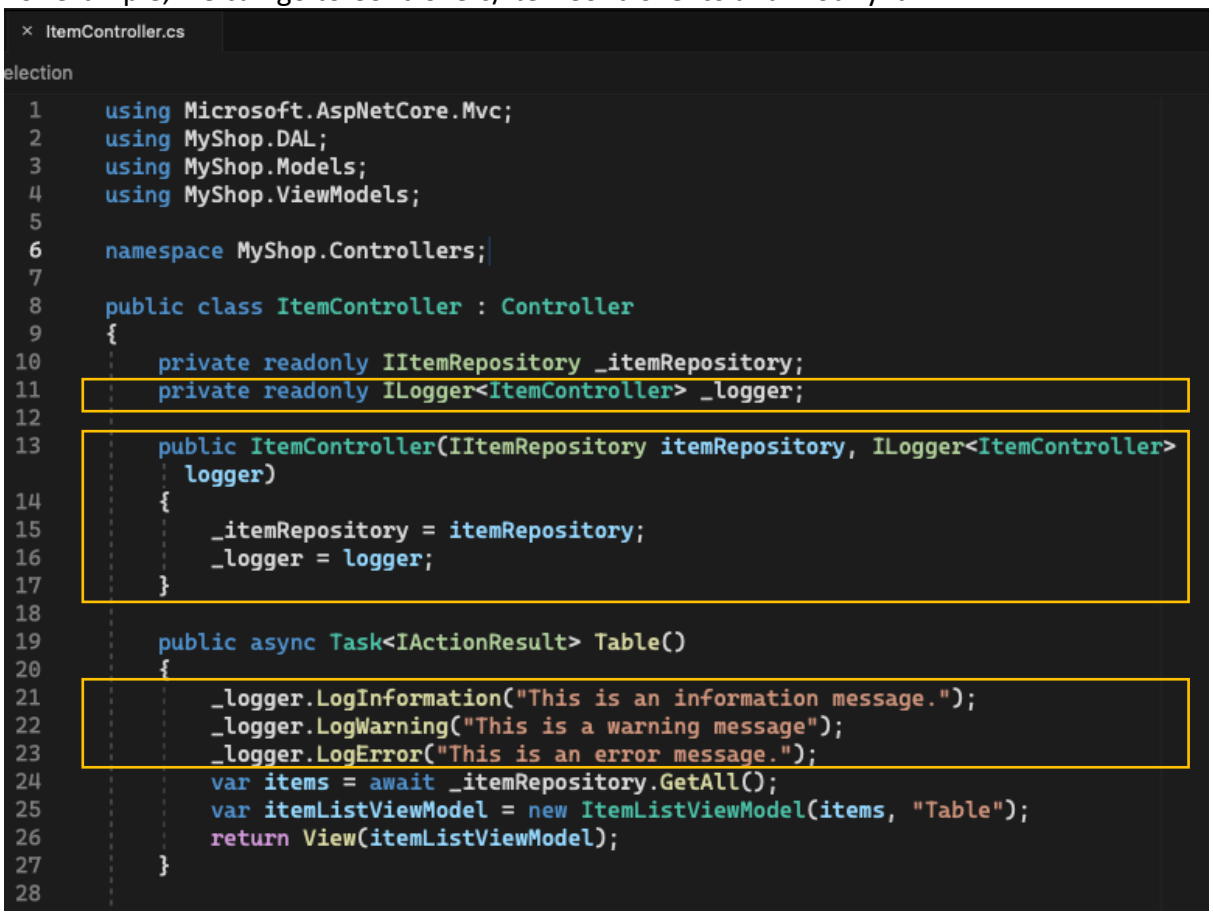
- **Line 3:** Import the Serilog namespace:
- **Lines 16–22:** Register Serilog with Dependency Injection (builder.Services.AddSerilog) and construct the configuration:
    - **Line 19 (.MinimumLevel.Information()):** Sets the minimum log level to Information. Any events lower than Information (such as Debug or Trace) will be ignored.
    - **Line 20 (.WriteTo.Console()):** Outputs color-coded log messages directly to the terminal for real-time debugging during development.
    - **Line 21 (.WriteTo.File(...)):** Generates a distinct .log file in the Logs/ directory every time the application starts, using the current date and timestamp (e.g., Logs/app_20260831_143000.log).



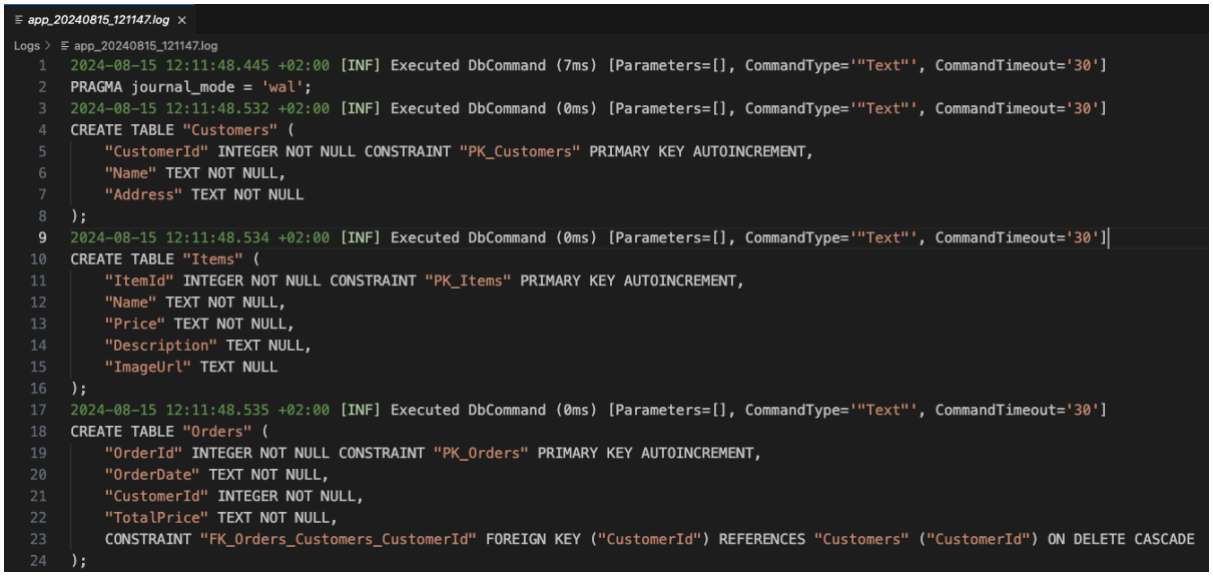
Logging Levels Overview
When creating custom logs, you can choose from different log levels depending on the severity of the event:

- **Trace:** Extremely detailed diagnostic information, typically used for deep debugging.
- **Debug:** Developer-focused internal system events and variable states.
- **Information:** General runtime events tracking normal application operation (e.g., "Item successfully created").
- **Warning:** Non-critical issues or unexpected behavior that do not stop execution.
- **Error:** Exceptions or failures in current operations (e.g., database connection dropped, failed updates).
- **Fatal (or Critical):** Severe failures causing application crashes or unrecoverable system states.

Now we have the logger. To use it for logging our customised message, we need to modify where it is used.
For example, we can go to Controllers/ItemController.cs and modify it:



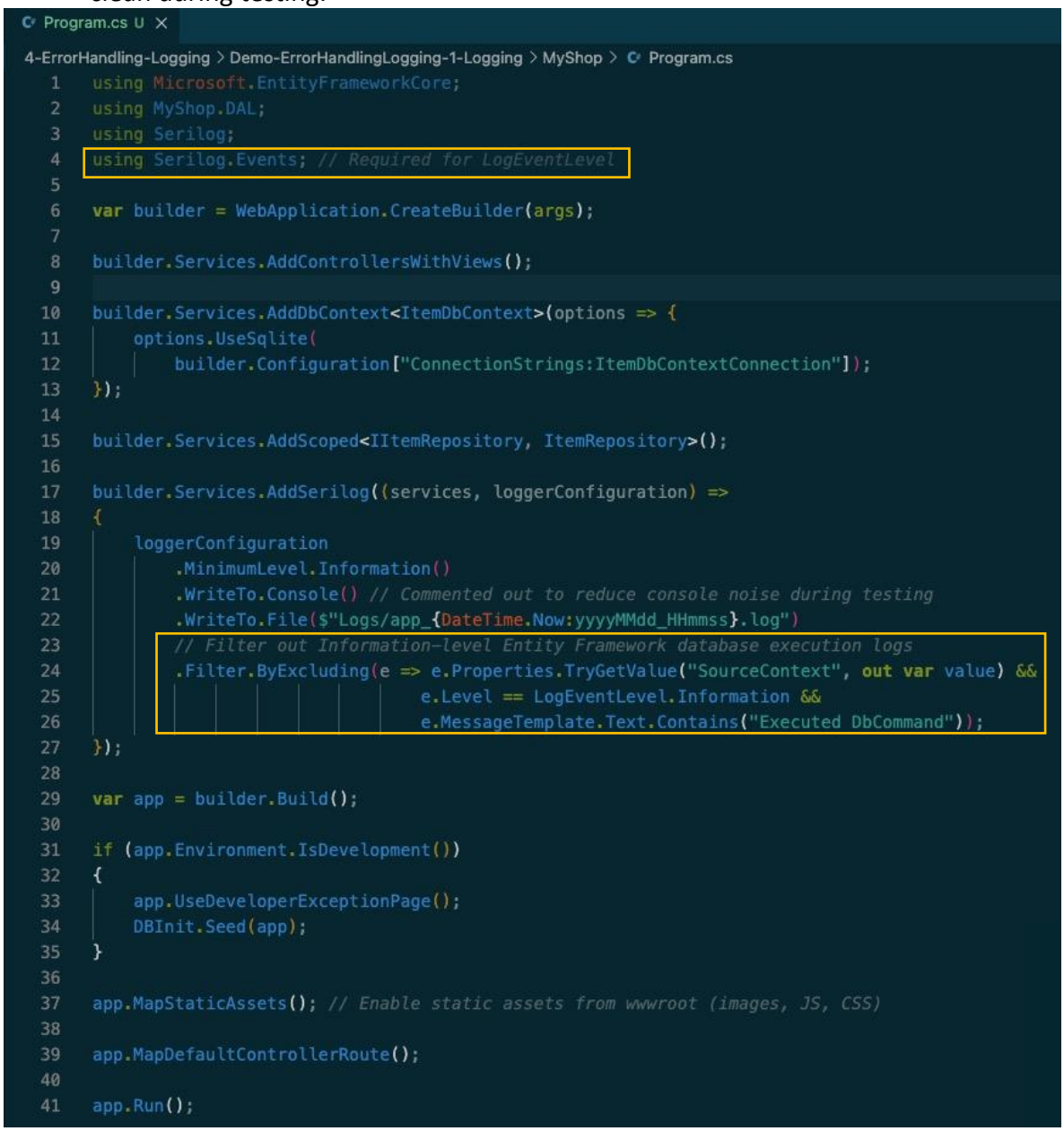
Run the project. Go to the folder Logs, open the logging file. We can see that the logging is working! But it is too verbose. (If it does not update, you may need to stop the project.)
If we observe, then we can find out many lines are about “Executed DbCommand”, which is pretty verbose:


app_20240815_121147.log
Logs > app_20240815_121147.log
2024-08-15 12:11:48.445 +02:00 [INF] Executed DbCommand (7ms) [Parameters=[], CommandType='"Text"', CommandTimeout='30']
PRAGMA journal_mode = 'wal';
2024-08-15 12:11:48.532 +02:00 [INF] Executed DbCommand (0ms) [Parameters=[], CommandType='"Text"', CommandTimeout='30']
CREATE TABLE "Customers" (
"CustomerId" INTEGER NOT NULL CONSTRAINT "PK_Customers" PRIMARY KEY AUTOINCREMENT,
"Name" TEXT NOT NULL,
"Address" TEXT NOT NULL
);
2024-08-15 12:11:48.534 +02:00 [INF] Executed DbCommand (0ms) [Parameters=[], CommandType='"Text"', CommandTimeout='30']
CREATE TABLE "Items" (
"ItemId" INTEGER NOT NULL CONSTRAINT "PK_Items" PRIMARY KEY AUTOINCREMENT,
"Name" TEXT NOT NULL,
"Price" TEXT NOT NULL,
"Description" TEXT NULL,
"ImageUrl" TEXT NULL
);
2024-08-15 12:11:48.535 +02:00 [INF] Executed DbCommand (0ms) [Parameters=[], CommandType='"Text"', CommandTimeout='30']
CREATE TABLE "Orders" (
"OrderId" INTEGER NOT NULL CONSTRAINT "PK_Orders" PRIMARY KEY AUTOINCREMENT,
"OrderDate" TEXT NOT NULL,
"CustomerId" INTEGER NOT NULL,
"TotalPrice" TEXT NOT NULL,
CONSTRAINT "FK_Orders_Customers_CustomerId" FOREIGN KEY ("CustomerId") REFERENCES "Customers" ("CustomerId") ON DELETE CASCADE
);

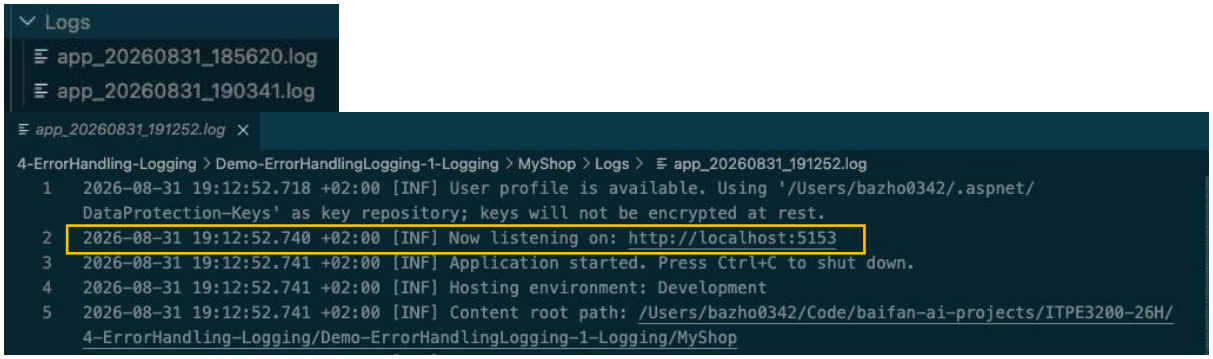
Filtering Logging Messages

To keep our log files clean and focused, we can filter out repetitive database queries generated by Entity Framework Core while keeping our custom application logs.
Modify Program.cs:

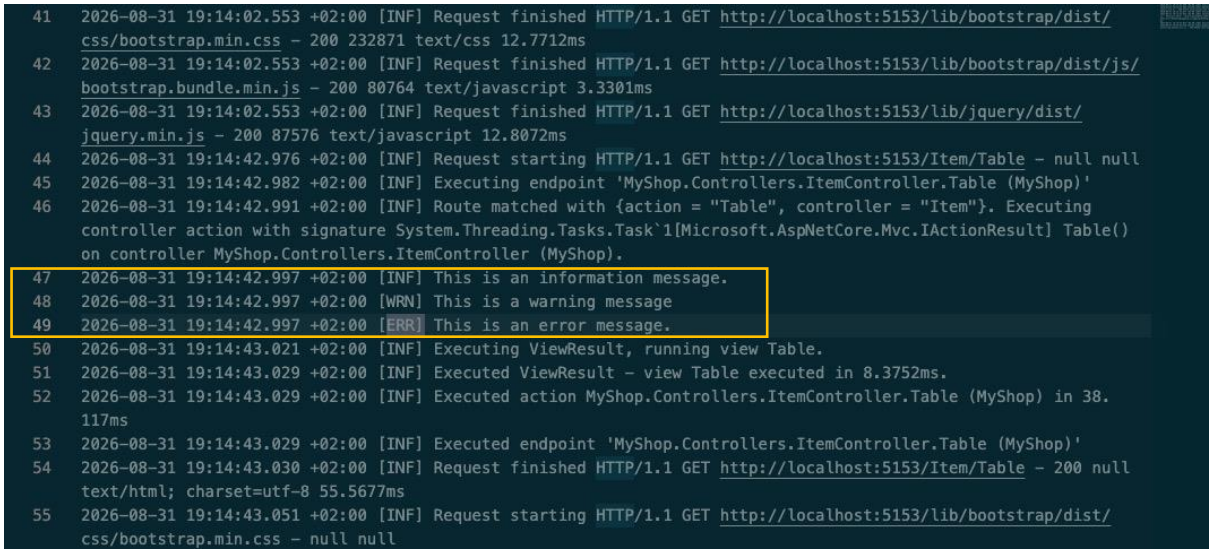
1. Add using Serilog.Events; at the top of the file (**Line 4**).
2. Add the .Filter.ByExcluding(...) configuration inside builder.Services.AddSerilog.
3. *(Optional)* Comment out .WriteTo.Console() if you want to keep the terminal window clean during testing.



Run the project.
If you commented out the WriteTo.Console(), you need to go to the newly generated log to access the localhost URL, since the console logs are output to the log file instead of the console.



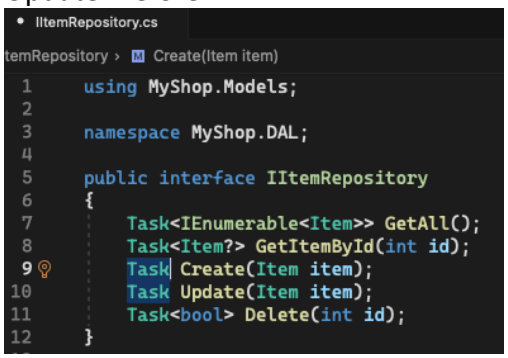
In the app, **Click Items** tab to invoke our customised logging information.
(If it does not update, you may need to wait a bit.)
Now the logging is much less verbose, and contains our customised information:



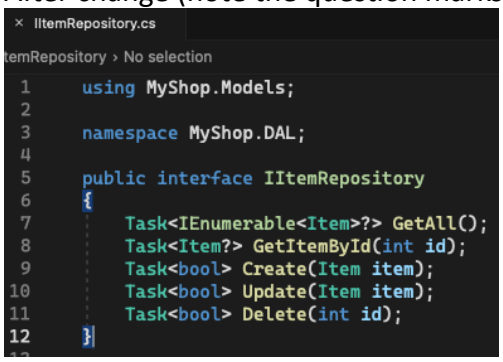
Error Handling

Error Handling - IItemRepository

Error handling has been partially covered before. This time we change the ItemRepository and all methods in Controllers/ItemController.cs to the version of handling errors.
First we go to DAL/IItemRepository.cs to modify the return types of GetAll, Create and Update. Before:



After change (note the question marks!):



Return Types for Write Operations (Create and Update)

- **Original:** Task Create(Item item) and Task Update(Item item)
- **Updated:** Task<bool> Create(Item item) and Task<bool> Update(Item item)

Why the change?

- **Confirmation of Success:** Void-returning tasks (Task) only tell the caller that the method finished without throwing an unhandled exception. Returning Task<bool> explicitly informs the calling controller whether the database persistence actually succeeded (returning true) or failed silently/gracefully (returning false).
- **Consistency:** It aligns Create and Update with Delete(int id), which already returns Task<bool>. Controllers can now use a unified pattern for all write/mutation actions:

Nullable Return Type for Read Operation (GetAll)

- **Original:** Task<IEnumerable<Item>> GetAll()
- **Updated:** Task<IEnumerable<Item>?> GetAll() (note the nullable ? modifier)

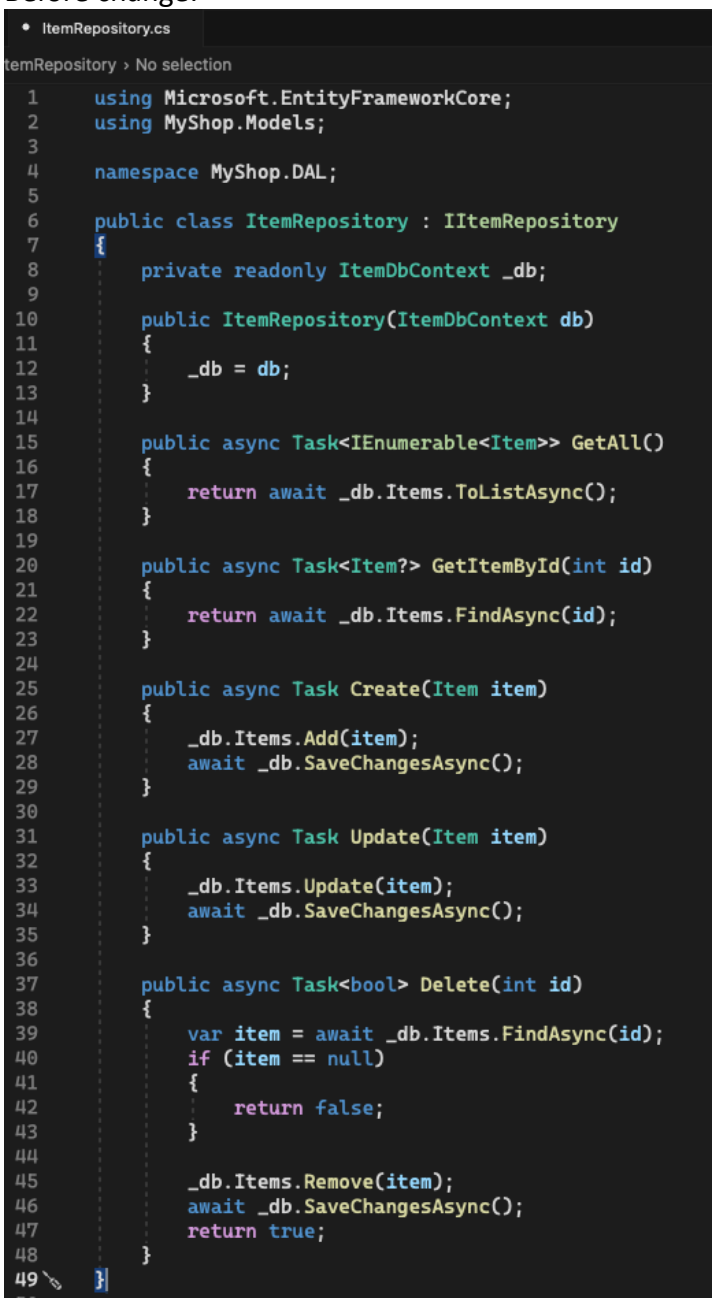
Why the change?

- **C# Nullable Reference Types Support:** Adding ? acknowledges that GetAll() might return null if a database connection error occurs or if the table cannot be queried (as opposed to returning an empty list [] when the table exists but contains no records).
- **Compiler Warning Prevention:** In modern C# (.NET 6+ with nullable checks enabled), marking the return type as IEnumerable<Item>? forces callers to write safety checks before accessing the collection.

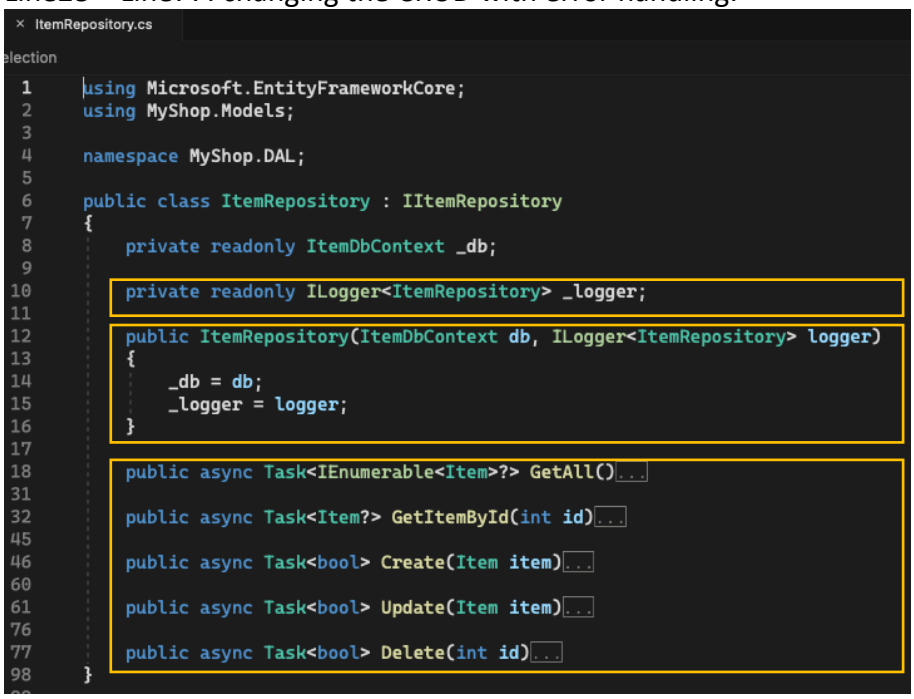
Error Handling - ItemRepository

Then we modify DAL/ItemRepository.cs

Before change:



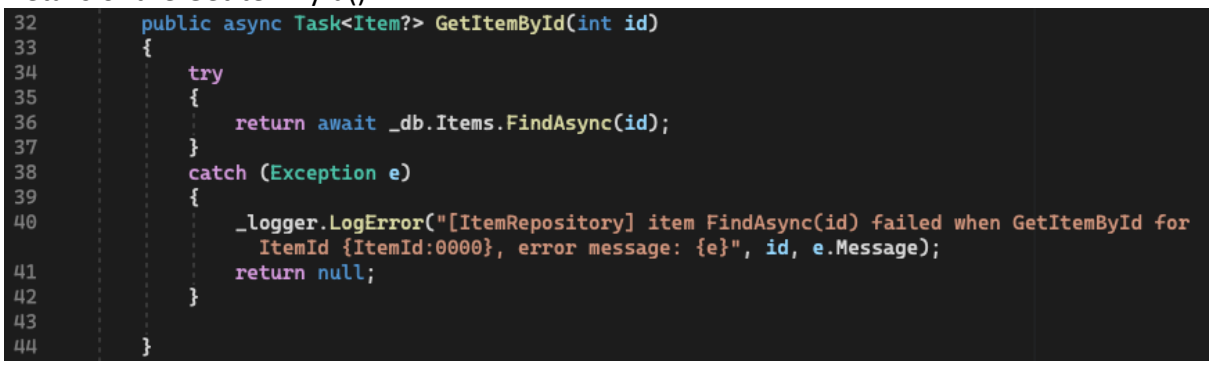
After change:
Line10: Adding a private readonly logger.
Line12: initialise the logger with the constructor.
Line28 – Line77: changing the CRUD with error handling.



Details of the GetAll():
Note that the error message has the tag [ItemController] indicating where the error comes from. Similar in all error messages in ItemRepository.cs



Details of the GetItemById():



Details of the Create():

```
46      public async Task<bool> Create(Item item)
47      {
48          try
49          {
50              _db.Items.Add(item);
51              await _db.SaveChangesAsync();
52              return true;
53          }
54          catch (Exception e)
55          {
56              _logger.LogError("[ItemRepository] item creation failed for item {@item}, error
                  message: {e}", item, e.Message);
57              return false;
58          }
59      }
```

Details of the Update():

```
61      public async Task<bool> Update(Item item)
62      {
63          try
64          {
65              _db.Items.Update(item);
66              await _db.SaveChangesAsync();
67              return true;
68          }
69          catch (Exception e)
70          {
71              _logger.LogError("[ItemRepository] item FindAsync(id) failed when updating the ItemId
                  {ItemId:0000}, error message: {e}", item, e.Message);
72              return false;
73          }
74
75      }
```

Details of the Delete():

```
77      public async Task<bool> Delete(int id)
78      {
79          try
80          {
81              var item = await _db.Items.FindAsync(id);
82              if (item == null)
83              {
84                  _logger.LogError("[ItemRepository] item not found for the ItemId {ItemId:0000}",
                      id);
85                  return false;
86              }
87
88              _db.Items.Remove(item);
89              await _db.SaveChangesAsync();
90              return true;
91          }
92          catch (Exception e)
93          {
94              _logger.LogError("[ItemRepository] item deletion failed for the ItemId {ItemId:0000},
                  error message: {e}", id, e.Message);
95              return false;
96          }
97      }
```

Error Handling - ItemController

Next we are going to add logging and error handling to ItemController.cs

You can see that the Details() method already partially covers error handling then the item of the given id is not found.

Before changing:

ItemController.cs

```
lection
1   using Microsoft.AspNetCore.Mvc;
2   using MyShop.DAL;
3   using MyShop.Models;
4   using MyShop.ViewModels;
5
6   namespace MyShop.Controllers;
7
8   public class ItemController : Controller
9   {
10      private readonly IItemRepository _itemRepository;
11      private readonly ILogger<ItemController> _logger;
12
13      public ItemController(IItemRepository itemRepository, ILogger<ItemController> logger)
14      {
15          _itemRepository = itemRepository;
16          _logger = logger;
17      }
18
19      public async Task<IActionResult> Table()
20      {
21          _logger.LogInformation("This is an information message.");
22          _logger.LogWarning("This is a warning message");
23          _logger.LogError("This is an error message.");
24          var items = await _itemRepository.GetAll();
25          var itemListViewModel = new ItemListViewModel(items, "Table");
26          return View(itemListViewModel);
27      }
28
29      public async Task<IActionResult> Grid()
30      {
31          var items = await _itemRepository.GetAll();
32          var itemListViewModel = new ItemListViewModel(items, "Grid");
33          return View(itemListViewModel);
34      }
35
36      public async Task<IActionResult> Details(int id)
37      {
38          var item = await _itemRepository.GetItemById(id);
39          if (item == null)
40              return BadRequest("Item not found.");
41          return View(item);
42      }
43
```

```
44          [HttpGet]
45          public IActionResult Create()
46          {
47              return View();
48          }
49
50          [HttpPost]
51          public async Task<IActionResult> Create(Item item)
52          {
53              if (ModelState.IsValid)
54              {
55                  await _itemRepository.Create(item);
56                  return RedirectToAction(nameof(Table));
57              }
58
59              return View(item);
60          }
61
62          [HttpGet]
63          public async Task<IActionResult> Update(int id)
64          {
65              var item = await _itemRepository.GetItemById(id);
66              if (item == null)
67              {
68                  return NotFound();
69              }
70              return View(item);
71          }
72
73          [HttpPost]
74          public async Task<IActionResult> Update(Item item)
75          {
76              if (ModelState.IsValid)
77              {
78                  await _itemRepository.Update(item);
79                  return RedirectToAction(nameof(Table));
80              }
81
82              return View(item);
83          }
84
85          [HttpGet]
86          public async Task<IActionResult> Delete(int id)
87          {
88              var item = await _itemRepository.GetItemById(id);
89              if (item == null)
90              {
91                  return NotFound();
92              }
93              return View(item);
94          }
95
96          [HttpPost]
97          public async Task<IActionResult> DeleteConfirmed(int id)
98          {
99              await _itemRepository.Delete(id);
100             return RedirectToAction(nameof(Table));
101         }
102     }
```

After changing:
Line11: adding a private readonly logger.
Line13 – Line17: initialise the logger in the constructor.
Line19 – Line109: the changed CRUD in ItemController with error handling and logging.

ItemController.cs

selection

```
1    using Microsoft.AspNetCore.Mvc;
2    using MyShop.DAL;
3    using MyShop.Models;
4    using MyShop.ViewModels;
5
6    namespace MyShop.Controllers;
7
8    public class ItemController : Controller
9    {
10       private readonly IItemRepository _itemRepository;
11       private readonly ILogger<ItemController> _logger;
12
13       public ItemController(IItemRepository itemRepository, ILogger<ItemController> logger)
14       {
15           _itemRepository = itemRepository;
16           _logger = logger;
17       }
18
19       public async Task<IActionResult> Table()...
30
31       public async Task<IActionResult> Grid()...
42
43       public async Task<IActionResult> Details(int id)...
53
54       [HttpGet]
55       public IActionResult Create()...
59
60       [HttpPost]
61       public async Task<IActionResult> Create(Item item)...
71
72       [HttpGet]
73       public async Task<IActionResult> Update(int id)...
83
84       [HttpPost]
85       public async Task<IActionResult> Update(Item item)...
95
96       [HttpGet]
97       public async Task<IActionResult> Delete(int id)...
107
108      [HttpPost]
109      public async Task<IActionResult> DeleteConfirmed(int id)...
119  }
```

Details of Read, including Table(), Grid(), Details():
Note that the error message has the tag [ItemController] indicating where the error comes from.

```
19      public async Task<IActionResult> Table()
20      {
21          var items = await _itemRepository.GetAll();
22          if (items == null)
23          {
24              _logger.LogError("[ItemController] Item list not found while executing _itemRepository.GetAll()");
25              return NotFound("Item list not found");
26          }
27          var itemsViewModel = new ItemsViewModel(items, "Table");
28          return View(itemsViewModel);
29      }
30
        0 references
31      public async Task<IActionResult> Grid()
32      {
33          var items = await _itemRepository.GetAll();
34          if (items == null)
35          {
36              _logger.LogError("[ItemController] Item list not found while executing _itemRepository.GetAll()");
37              return NotFound("Item list not found");
38          }
39          var itemsViewModel = new ItemsViewModel(items, "Grid");
40          return View(itemsViewModel);
41      }
42
        0 references
43      public async Task<IActionResult> Details(int id)
44      {
45          var item = await _itemRepository.GetItemById(id);
46          if (item == null)
47          {
48              _logger.LogError("[ItemController] Item not found for the ItemId {ItemId:0000}", id);
49              return NotFound("Item not found for the ItemId");
50          }
51          return View(item);
52      }
```

Details of Create:

```
54          [HttpGet]
55          public IActionResult Create()
56          {
57              return View();
58          }
59
60          [HttpPost]
61          public async Task<IActionResult> Create(Item item)
62          {
63              if (ModelState.IsValid)
64              {
65                  bool returnOk = await _itemRepository.Create(item);
66                  if (returnOk)
67                      return RedirectToAction(nameof(Table));
68              }
69              _logger.LogWarning("[ItemController] Item creation failed {@item}", item);
70              return View(item);
71          }
```

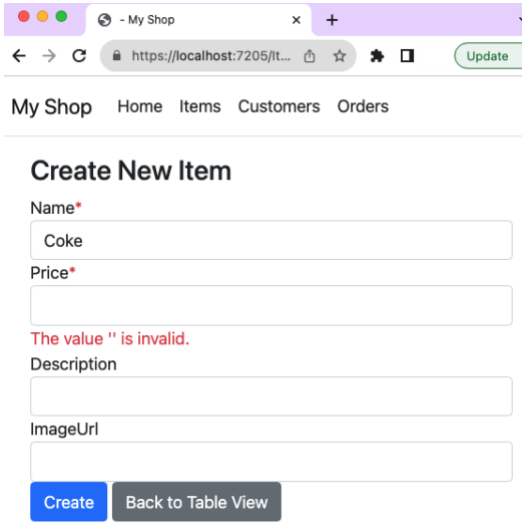
Details of Update:

```
73      [HttpGet]
74      public async Task<IActionResult> Update(int id)
75      {
76          var item = await _itemRepository.GetItemById(id);
77          if (item == null)
78          {
79              _logger.LogError("[ItemController] Item not found when updating the ItemId {ItemId:0000}", id);
80              return BadRequest("Item not found for the ItemId");
81          }
82          return View(item);
83      }
84
85      [HttpPost]
86      public async Task<IActionResult> Update(Item item)
87      {
88          if (ModelState.IsValid)
89          {
90              bool returnOk = await _itemRepository.Update(item);
91              if (returnOk)
92                  return RedirectToAction(nameof(Table));
93          }
94          _logger.LogWarning("[ItemController] Item update failed {@item}", item);
95          return View(item);
96      }
```

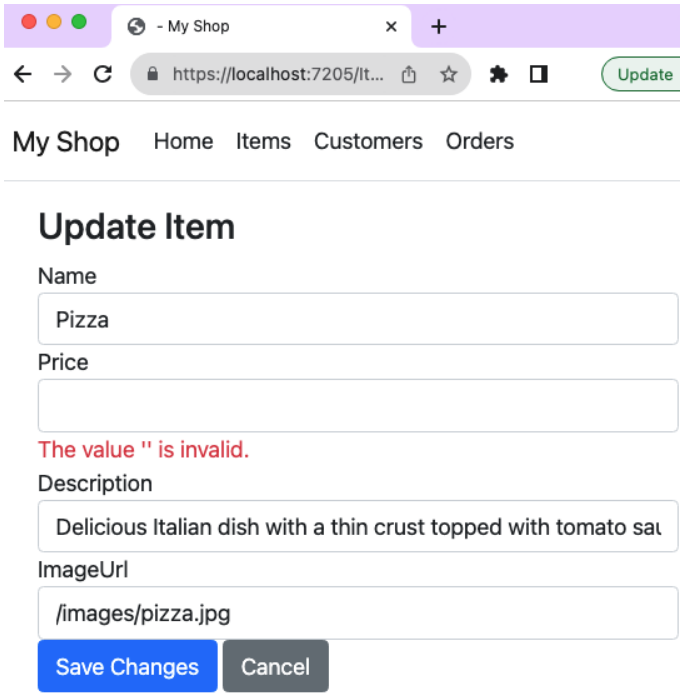
Details of Delete:

```
 98      [HttpGet]
 99      public async Task<IActionResult> Delete(int id)
100      {
101          var item = await _itemRepository.GetItemById(id);
102          if (item == null)
103          {
104              _logger.LogError("[ItemController] Item not found for the ItemId {ItemId:0000}", id);
105              return BadRequest("Item not found for the ItemId");
106          }
107          return View(item);
108      }
109
110      [HttpPost]
111      public async Task<IActionResult> DeleteConfirmed(int id)
112      {
113          bool returnOk = await _itemRepository.Delete(id);
114          if (!returnOk)
115          {
116              _logger.LogError("[ItemController] Item deletion failed for the ItemId {ItemId:0000}", id);
117              return BadRequest("Item deletion failed");
118          }
119          return RedirectToAction(nameof(Table));
120      }
```

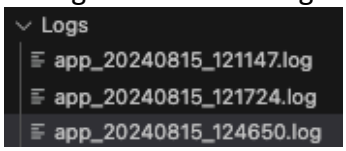
Now run the project.
Go to Items -> Create New Item. We make an error by not inputting Price and click Create.



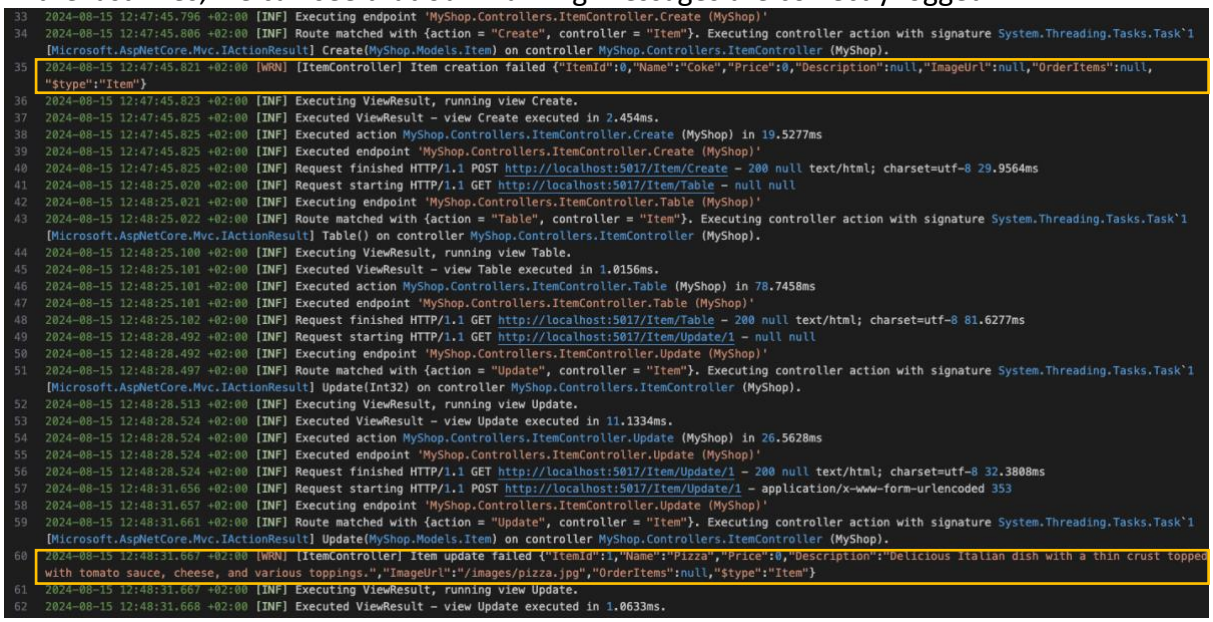
We also go to Update one item and delete its price, click Save Changes.



Now go to the latest log files under Logs/. (If it does not update, you may need to wait a bit)



In the last lines, we can see that our Warning Messages are correctly logged:



Congratulations! ☺ You have a good web application with multi-layers of logging and error handling.